PRACTICAL 3-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 3 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Distinct Points, Duplicate Cards, Fractions and Part B: Object methods and nested Address.

CODE SCREENSHOT:

Card.java

Part A > Practical 3-A > Card.java > ...

```java
import java.util.Objects;

public class Card {
    private final String rank, suit;
nshul\Downloads\OOP_Practicals_1-13_A_B\Part A\Practical 3-A\Card.java
    t) {
        this.rank = rank;
        this.suit = suit;
    }

    @Override
    public String toString() {
        return rank + " of " + suit;
    }

    @Override
    public boolean equals(Object o) {
        return o instanceof Card c && rank.equals(c.rank) && suit.equals(c.suit);
    }

    @Override
    public int hashCode() {
        return Objects.hash(rank, suit);
    }
}
```

Code Screenshot 1

CardDriver.java

Part A > Practical 3-A > CardDriver.java > ...

```java
import java.util.*;

public class CardDriver {
    Run | Debug
    public static void main(String[] args) {
        Card[] cards = { new Card(rank: "Ace", suit: "Spades"), new Card(rank: "King", suit: "Hearts"), new Card(rank: "Ace", suit: "Spades") };
        List<Card> seen = new ArrayList<>();
        for (Card c : cards) {
            if (seen.contains(c)) {
                System.out.println("Duplicate found: " + c);
                break;
            }
            seen.add(c);
        }
    }
}
```

Code Screenshot 2

Driver.java

Part A > Practical 3-A > Driver.java > ...

```java
import java.util.*;

public class Driver {
    Run | Debug
    public static void main(String[] args) {
        Point[] points = { new Point(x: 1, y: 2), new Point(x: 2, y: 3), new Point(x: 1, y: 2), new Point(x: 4, y: 5), new Point(x: 2, y: 3) };
        List<Point> distinct = new ArrayList<>();
        for (Point p : points)
            if (!distinct.contains(p))
                distinct.add(p);
        System.out.println("Distinct: " + distinct.size());
    }
}

```

Code Screenshot 3

Fraction.java

Part A > Practical 3-A > Fraction.java > ...

```java
import java.util.Objects;

public class Fraction {
    private final int num, den;

    public Fraction(int num, int den) {
        if (den == 0)
            throw new IllegalArgumentException("Denominator cannot be zero");
        if (den < 0) {
            num = -num;
            den = -den;
        }
        int g = gcd(Math.abs(num), den);
        this.num = num / g;
        this.den = den / g;
    }

    private static int gcd(int a, int b) {
        while (b != 0) {
            int t = a % b;
            a = b;
            b = t;
        }
        return a == 0 ? 1 : a;
    }

    @Override
    public String toString() {
        return num + "/" + den;
    }

    @Override
    public boolean equals(Object o) {
        return o instanceof Fraction f && num == f.num && den == f.den;
    }

    @Override
    public int hashCode() {
        return Objects.hash(num, den);
    }

    Run | Debug
    public static void main(String[] args) {
        Fraction a = new Fraction(num: 2, den: 4), b = new Fraction(num: 1, den: 2);
        System.out.println(a);
        System.out.println("Equal: " + a.equals(b));
    }
}
```

Code Screenshot 4

Point.java

Part A > Practical 3-A > Point.java > ...

```java
import java.util.Objects;

public class Point {
    private final int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }

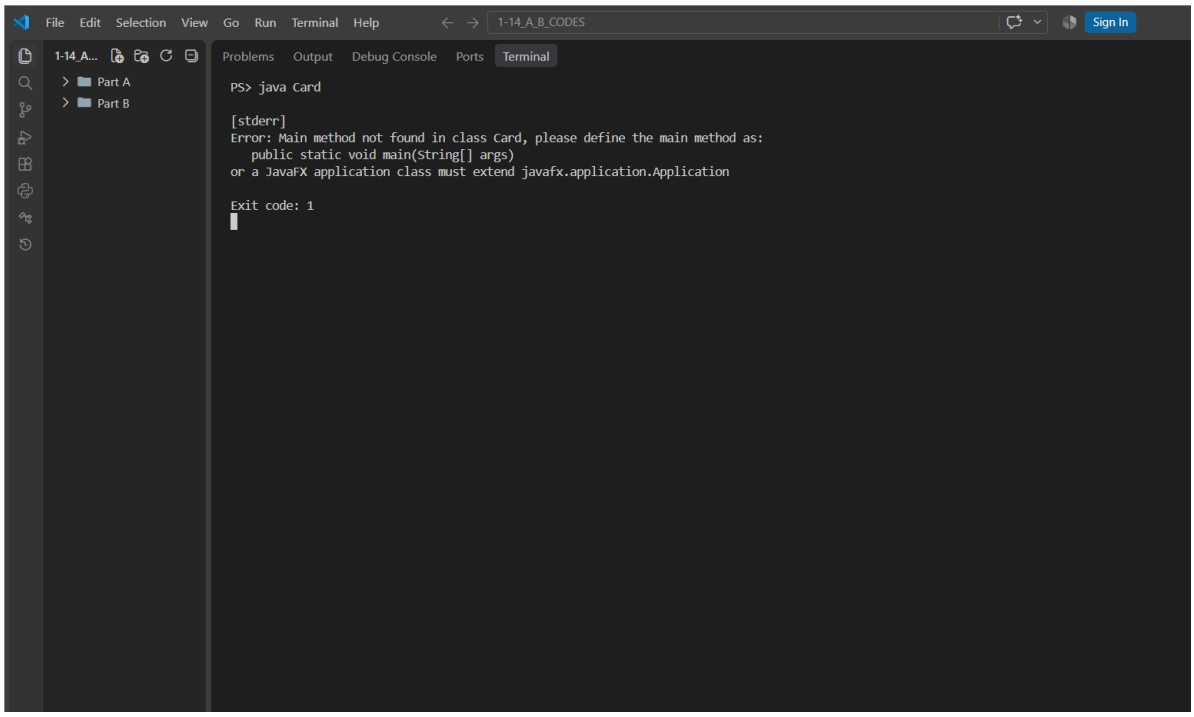
    @Override
    public boolean equals(Object o) {
        return o instanceof Point p && x == p.x && y == p.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }
}

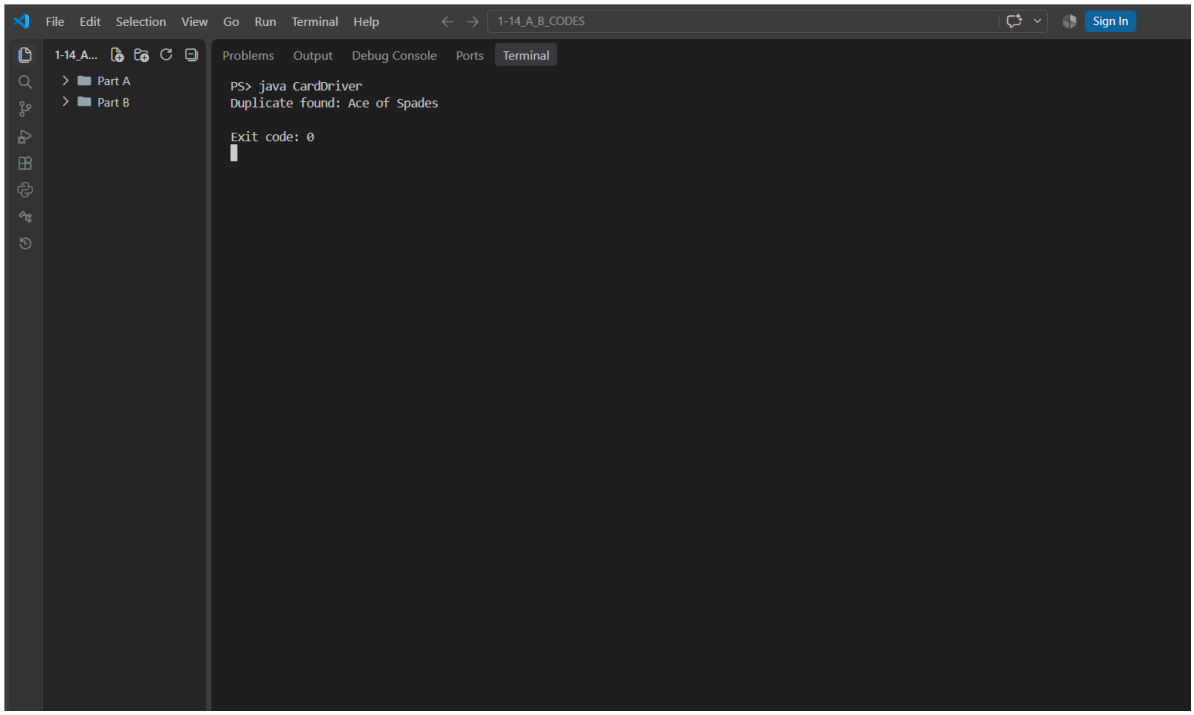
```

Code Screenshot 5

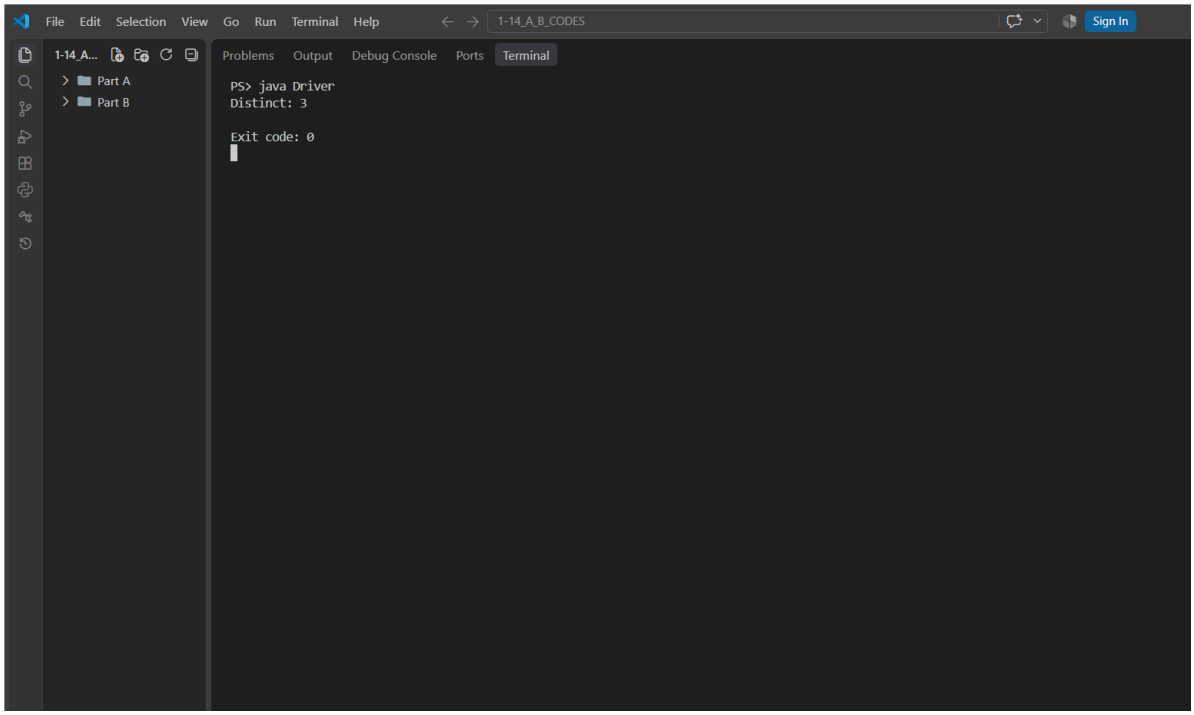
OUTPUT SCREENSHOT:



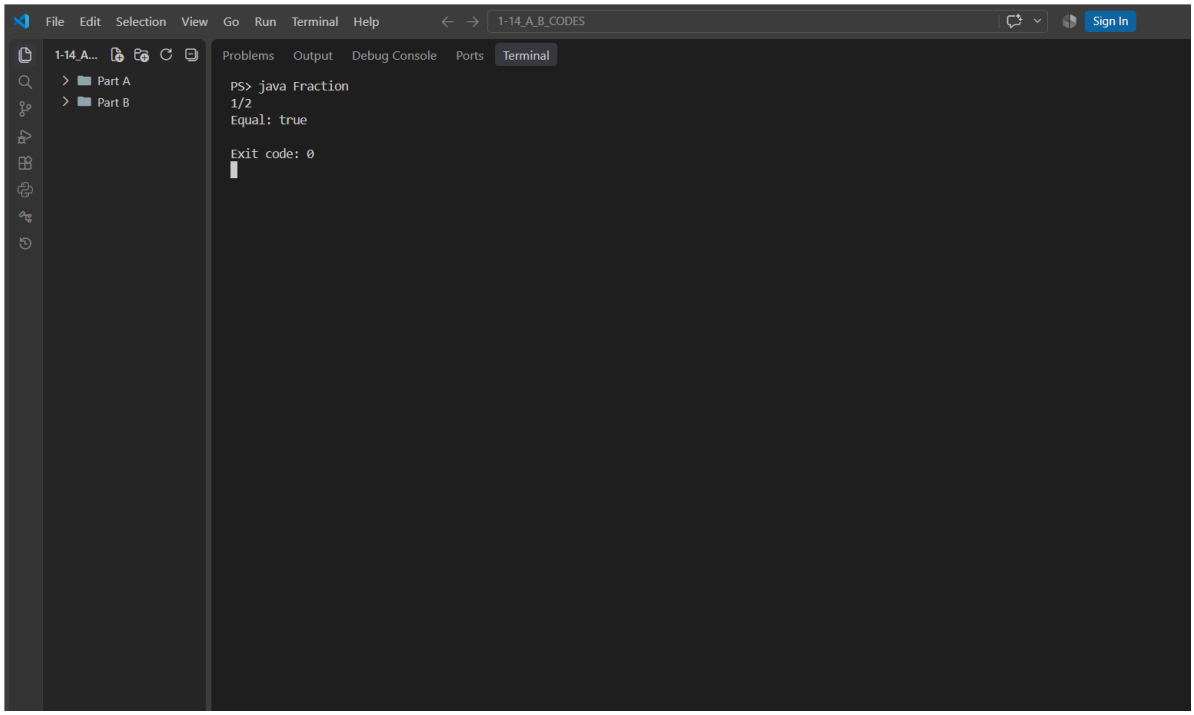
Output Screenshot 1



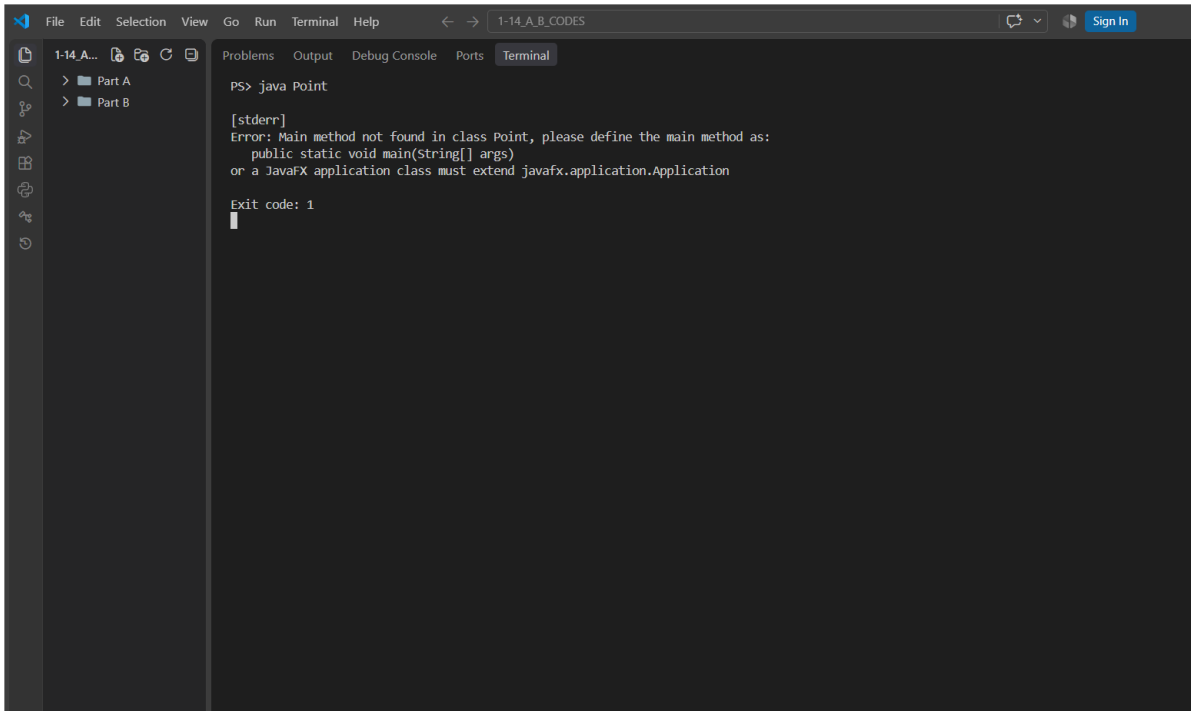
Output Screenshot 2



Output Screenshot 3



Output Screenshot 4



Output Screenshot 5

LEARNING OUTCOMES:

Override Object methods correctly and use equality, hashing, nested classes, and object identity checks.